

Intro Data Manipulation with dplyr

So far, most of the datasets we have worked with have been prepared so that we can immediately begin an analysis. When we wanted to create a visualization, for example, the observations and variables we needed were usually already present in the data frame in a useful form.

Real data will not always be organized for the particular question we want to answer. A data frame may contain observations that are not relevant to our question or variables that we do not need. The quantity we want to study may not be recorded directly and instead need to be calculated from other variables. Sometimes the information we need may even be spread across several observations or several data frames.

Understanding the Data in Front of Us

Before deciding how to manipulate a dataset, we first need to understand what the data represent (and thus what we have access to).

Recall that the *columns* of a data frame represent *variables*. Each variable records some characteristic or measurement. The *rows* represent *observations*. What constitutes one observation, however, deserves some care.

Consider the following observations from the `Batting` dataset in the `Lahman` package. These are actual batting records for Manny Ramirez from 2007 through 2009.

##	year	team	games	home_runs
## 1	2007	BOS	133	20
## 2	2008	BOS	100	20
## 3	2008	LAN	53	17
## 4	2009	LAN	104	19

It might initially seem reasonable to say that each row represents a player during a season. However, Ramirez appears twice in 2008 because he played for both Boston and Los Angeles that year.

In these data, one observation represents *one player's record for one team during one season*.

This affects what questions we can answer directly from a row. For example, we can see how many home runs Ramirez hit for Boston in 2008 and how many he hit for Los Angeles. However, no single row tells us how many home runs he hit during the entire 2008 season. The information needed to answer that question is present in the data, but it is spread across multiple observations.

As we begin manipulating data, we need to keep two questions in mind:

- What does one row represent?**
- What does each column represent?**

The answers help determine what we can currently learn from the data and what we might need to change.

Data Manipulation with dplyr

Today we will begin working with `dplyr`, a package designed for manipulating data frames.

We will focus on four common `dplyr` functions:

Function	Purpose
<code>filter()</code>	Choose observations
<code>select()</code>	Choose variables
<code>mutate()</code>	Create or modify variables
<code>arrange()</code>	Change the order of observations

The Pipe

Data manipulation often requires us to perform several operations in sequence. We will use the **pipe operator**, `|>`, to make these sequences easier to read.

For example:

```
penguins |>
  filter(species == "Gentoo")

## # A tibble: 124 x 8
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>             <int>         <int>
## 1 Gentoo  Biscoe          46.1           13.2              211          4500
## 2 Gentoo  Biscoe           50           16.3              230          5700
## 3 Gentoo  Biscoe          48.7           14.1              210          4450
## 4 Gentoo  Biscoe           50           15.2              218          5700
## 5 Gentoo  Biscoe          47.6           14.5              215          5400
## 6 Gentoo  Biscoe          46.5           13.5              210          4550
## 7 Gentoo  Biscoe          45.4           14.6              211          4800
## 8 Gentoo  Biscoe          46.7           15.3              219          5200
## 9 Gentoo  Biscoe          43.3           13.4              209          4400
## 10 Gentoo Biscoe          46.8           15.4              215          5150
## # i 114 more rows
## # i 2 more variables: sex <fct>, year <int>
```

We can read this from left to right as:

Start with the `penguins` data, **then** retain the Gentoo penguins.

As we perform more operations, we can add additional steps to the pipeline. We will use this structure throughout the lab.

Getting Started

We will use several R packages and datasets throughout today's lab.

```
# install.packages(c("dplyr", "palmerpenguins", "dslabs", "gapminder"))
library(dplyr)
library(palmerpenguins)
library(dslabs)
library(gapminder)
```

We will also use a dataset containing information about colleges and universities from the 2019 scorecard.

```
colleges = read.csv("https://remiller1450.github.io/data/Colleges2019.csv")
```

Choosing Observations with `filter()`

A dataset may contain many observations even when our question concerns only some of them. The `filter()` function allows us to retain observations that satisfy conditions we specify.

For example, the `penguins` dataset contains measurements of penguins observed on three islands in Antarctica. Each row represents one penguin.

Suppose we are interested specifically in Gentoo penguins:

```
penguins |>
  filter(species == "Gentoo")

## # A tibble: 124 x 8
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>             <int>         <int>
## 1 Gentoo  Biscoe          46.1          13.2              211          4500
## 2 Gentoo  Biscoe          50           16.3              230          5700
## 3 Gentoo  Biscoe          48.7          14.1              210          4450
## 4 Gentoo  Biscoe          50           15.2              218          5700
## 5 Gentoo  Biscoe          47.6          14.5              215          5400
## 6 Gentoo  Biscoe          46.5          13.5              210          4550
## 7 Gentoo  Biscoe          45.4          14.6              211          4800
## 8 Gentoo  Biscoe          46.7          15.3              219          5200
## 9 Gentoo  Biscoe          43.3          13.4              209          4400
## 10 Gentoo Biscoe          46.8          15.4              215          5150
## # i 114 more rows
## # i 2 more variables: sex <fct>, year <int>
```

The condition inside `filter()` determines which observations are retained. Observations for which the condition is `TRUE` are retained, while observations for which it is `FALSE` are removed.

Logical Conditions

The conditions used inside `filter()` are the same **logical conditions** that we have used previously in R. We are simply using the resulting `TRUE` and `FALSE` values to determine which observations should remain in our data.

Recall some of the logical operators we have used:

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code> , <code><=</code>	less than, less than or equal to
<code>></code> , <code>>=</code>	greater than, greater than or equal to
<code>&</code>	AND
<code> </code>	OR
<code>!</code>	NOT
<code>%in%</code>	belongs to a collection of values

For example, we could retain penguins observed on either Dream or Biscoe Island using:

```
penguins |>
  filter(island %in% c("Dream", "Biscoe"))
```

```
## # A tibble: 292 x 8
```

```
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>             <int>      <int>
##  1 Adelie  Biscoe         37.8           18.3             174       3400
##  2 Adelie  Biscoe         37.7           18.7             180       3600
##  3 Adelie  Biscoe         35.9           19.2             189       3800
##  4 Adelie  Biscoe         38.2           18.1             185       3950
##  5 Adelie  Biscoe         38.8           17.2             180       3800
##  6 Adelie  Biscoe         35.3           18.9             187       3800
##  7 Adelie  Biscoe         40.6           18.6             183       3550
##  8 Adelie  Biscoe         40.5           17.9             187       3200
##  9 Adelie  Biscoe         37.9           18.6             172       3150
## 10 Adelie  Biscoe         40.5           18.9             180       3950
## # i 282 more rows
## # i 2 more variables: sex <fct>, year <int>
```

The `%in%` operator is particularly convenient when several values of the **same variable** are acceptable.

We can also combine conditions. Suppose we want Gentoo penguins weighing more than 5000 grams:

```
penguins |>
  filter(
    species == "Gentoo",
    body_mass_g > 5000
  )
```

```
## # A tibble: 61 x 8
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>             <int>      <int>
##  1 Gentoo  Biscoe         50           16.3             230       5700
##  2 Gentoo  Biscoe         50           15.2             218       5700
##  3 Gentoo  Biscoe         47.6          14.5             215       5400
##  4 Gentoo  Biscoe         46.7          15.3             219       5200
##  5 Gentoo  Biscoe         46.8          15.4             215       5150
##  6 Gentoo  Biscoe         49           16.1             216       5550
##  7 Gentoo  Biscoe         48.4          14.6             213       5850
##  8 Gentoo  Biscoe         49.3          15.7             217       5850
##  9 Gentoo  Biscoe         49.2          15.2             221       6300
## 10 Gentoo  Biscoe         48.7          15.1             222       5350
## # i 51 more rows
## # i 2 more variables: sex <fct>, year <int>
```

When multiple conditions are separated by commas inside `filter()`, all of the conditions must be satisfied. This means we don't need to use a lot of `&s` if we are applying multiple conditions.

Question #1

A. Suppose we are interested specifically in the penguins observed on **Dream Island**. Use `filter()` to create a data frame containing these observations.

B. Smol or Chonkin': Suppose we are interested in penguins with particularly extreme body masses. Create a data frame containing penguins observed on Dream Island that weigh either less than 3000 grams or more than 4500 grams.

C. Without running any additional code, suppose we first filtered the data to contain all Adelie penguins and then added the condition that the penguins must have been observed on Dream Island. Could adding this condition increase the number of observations? Explain.

More Practice with `filter()`

Let's practice filtering in a different context. The `colleges` data contain information about colleges and universities from the 2019 College Scorecard.

```
head(colleges)
```

```
##              Name              City State Enrollment Private
## 1 Abilene Christian University    Abilene    TX      3524 Private
## 2   Academy of Art University San Francisco    CA      6898 Private
## 3      Adelphi University    Garden City    NY      5307 Private
## 4      Adrian College          Adrian    MI      1781 Private
## 5 AdventHealth University      Orlando    FL      1166 Private
## 6   Agnes Scott College      Decatur    GA        966 Private
##      Region Adm_Rate ACT_median ACT_Q1 ACT_Q3  Cost Net_Tuition
## 1 South West   0.5696         24     21     21 48046      16177
## 2   Far West        NA         NA     NA     NA 39735      31908
## 3   Mid East   0.7418         25     22     22 49008      24971
## 4 Great Lakes   0.6481         23     19     19 51626      14136
## 5 South East   0.8689         20     18     18 24338      15360
## 6 South East   0.7040         NA     NA     NA 52309      13239
##      Avg_Fac_Salary PercentFemale PercentWhite PercentBlack PercentHispanic
## 1          69804      0.6118200      0.7946      0.0814      0.1635
## 2          76428      0.5731982      0.6791      0.0825      0.1534
## 3         111339      0.7211121      0.6669      0.1785      0.1292
## 4          72873      0.4221106      0.8861      0.0692      0.0318
## 5          69759      0.8251058      0.7622      0.1395      0.1338
## 6          82647         NA      0.6884      0.2299      0.0665
##      PercentAsian FourYearComp_Males FourYearComp_Females Debt_median
## 1          0.0287      0.41157556      0.5283019      16000
## 2          0.1191      0.09157212      0.1325224      12877
## 3          0.0673      0.61146497      0.6998855      19500
## 4          0.0121      0.23209169      0.3319838      18468
## 5          0.0259      0.47619048      0.4132231      16646
## 6          0.0321         NA      NA      22450
##      Salary10yr_median
## 1          43000
## 2          38600
## 3          58500
## 4          38600
## 5          56000
## 6          39400
```

Take a moment to examine the variables contained in these data in your own **RStudio** before continuing (yes, I know we've seen this before)

Question #2

For each situation below, use `filter()` and appropriate logical conditions to create the requested data.

A. A student wants to investigate colleges in **Iowa** where the median ACT score is at least 25. Create a data frame containing colleges satisfying both conditions.

B. A student would be happy attending a college located in **Iowa, Minnesota, or Wisconsin**. Create a data frame containing colleges from any of these three states.

C. A student is interested in colleges that either have an admission rate below 50% **or** have a median ACT

score of at least 30. Create a data frame containing colleges satisfying at least one of these requirements.

D. Suppose a student is looking for colleges with an annual cost below \$50,000, an average net tuition below \$25,000, and a median student debt below \$20,000. Use a single `filter()` to find colleges satisfying all three requirements. Then perform the same filtering using three `filter()` calls in sequence, applying one requirement at a time. Verify that both approaches result in the same number of colleges.

Choosing Variables with `select()`

While `filter()` chooses observations, `select()` chooses **variables**.

Suppose we are interested only in penguin species, body mass, and flipper length:

```
penguins |>
  select(species, body_mass_g, flipper_length_mm)
```

```
## # A tibble: 344 x 3
##   species body_mass_g flipper_length_mm
##   <fct>      <int>          <int>
## 1 Adelie    3750            181
## 2 Adelie    3800            186
## 3 Adelie    3250            195
## 4 Adelie     NA             NA
## 5 Adelie    3450            193
## 6 Adelie    3650            190
## 7 Adelie    3625            181
## 8 Adelie    4675            195
## 9 Adelie    3475            193
## 10 Adelie   4250            190
## # i 334 more rows
```

The resulting data contain the same observations, but only the three variables we selected.

Unlike `filter()`, `select()` is often not necessary to perform an analysis. R can usually work with a data frame even when it contains variables that we do not need. Instead, `select()` is useful for creating a simpler data frame containing only the information relevant to our current task and making the results easier to read.

We can also remove variables by placing `-` before their names:

```
penguins |>
  select(-year)
```

```
## # A tibble: 344 x 7
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>      <dbl>         <dbl>          <int>      <int>
## 1 Adelie Torgersen    39.1          18.7            181       3750
## 2 Adelie Torgersen    39.5          17.4            186       3800
## 3 Adelie Torgersen    40.3           18            195       3250
## 4 Adelie Torgersen     NA           NA             NA         NA
## 5 Adelie Torgersen    36.7          19.3            193       3450
## 6 Adelie Torgersen    39.3          20.6            190       3650
## 7 Adelie Torgersen    38.9          17.8            181       3625
## 8 Adelie Torgersen    39.2          19.6            195       4675
## 9 Adelie Torgersen    34.1          18.1            193       3475
## 10 Adelie Torgersen    42           20.2            190       4250
## # i 334 more rows
## # i 1 more variable: sex <fct>
```

Question #3

A. Suppose you are preparing a dataset for someone interested in comparing the **bill dimensions of the three penguin species**. Use `select()` to create a data frame containing only the variables relevant to this investigation.

B. Create a version of penguins containing every variable except `year` and `sex`.

C. Now suppose we are interested only in the body mass and flipper length of **Gentoo penguins**. Create a data frame containing the Gentoo observations and only the variables `species`, `body_mass_g`, and `flipper_length_mm`.

D. The original penguins data contain 344 observations and 8 variables. If we use `select()` with three variables, how many observations and variables should the resulting data contain?

Creating Variables with `mutate()`

Sometimes the quantity we want to study is not directly recorded as a variable, even though the information needed to calculate it is available.

The `mutate()` function allows us to create new variables using variables already contained in a data frame.

For example, body mass in penguins is recorded in grams. We could create body mass in kilograms:

```
penguins |>
  mutate(body_mass_kg = body_mass_g / 1000)

## # A tibble: 344 x 9
##   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
## 1 Adelie  Torgersen         39.1          18.7          181          3750
## 2 Adelie  Torgersen         39.5          17.4          186          3800
## 3 Adelie  Torgersen         40.3           18          195          3250
## 4 Adelie  Torgersen          NA           NA           NA           NA
## 5 Adelie  Torgersen         36.7          19.3          193          3450
## 6 Adelie  Torgersen         39.3          20.6          190          3650
## 7 Adelie  Torgersen         38.9          17.8          181          3625
## 8 Adelie  Torgersen         39.2          19.6          195          4675
## 9 Adelie  Torgersen         34.1          18.1          193          3475
## 10 Adelie Torgersen         42           20.2          190          4250
## # i 334 more rows
## # i 3 more variables: sex <fct>, year <int>, body_mass_kg <dbl>
```

The new variable is added to the data frame while the existing variables remain.

Question #4

A. Penguin bill length is recorded in millimeters. Use `mutate()` to create a new variable called `bill_length_cm` containing bill length measured in centimeters.

B. We can create multiple variables within the same call to `mutate()`. Modify your code from Part A to create both `bill_length_cm` and `bill_depth_cm`.

C. New variables do not have to be quantitative. Use `mutate()` and `ifelse()` to create a variable called `heavy_penguin` (or `Chonkers`) that takes the value “Yes” when a penguin weighs at least 4000 grams and “No” otherwise.

The examples above created new variables by transforming information already contained in the data. More importantly, `mutate()` often allows us to construct new information that was not directly recorded in the

original data by combining or transforming existing variables.

Creating a Rate

Consider the `murders` dataset:

```
head(murders)
```

```
##      state abb region population total
## 1  Alabama  AL  South   4779736   135
## 2  Alaska   AK   West    710231    19
## 3  Arizona  AZ   West   6392017   232
## 4  Arkansas AR  South   2915918    93
## 5 California CA  West  37253956  1257
## 6  Colorado CO  West   5029196    65
```

This dataset contains data on gun-related homicides in the United States during 2010.

Question #5

- A. What does one observation in `murders` represent?
- B. State populations vary considerably. Why does this make the total number of homicides difficult to use when comparing homicide prevalence across states?

One way to account for this difference in populations is to calculate the number of murders per 100,000 residents:

```
murders |>
  mutate(rate = total / population * 100000) |> head()
```

```
##      state abb region population total    rate
## 1  Alabama  AL  South   4779736   135 2.824424
## 2  Alaska   AK   West    710231    19 2.675186
## 3  Arizona  AZ   West   6392017   232 3.629527
## 4  Arkansas AR  South   2915918    93 3.189390
## 5 California CA  West  37253956  1257 3.374138
## 6  Colorado CO  West   5029196    65 1.292453
```

- C. What does a value of `rate = 5` mean in the context of these data?
- D. In one series of pipes, create a data frame containing only the state name, total number of murders, and the newly calculated murder rate. Show the first 6 rows of this data frame.
- E. What could go wrong if you applied `select()` before `mutate()` in Part D? Explain why the order of these operations matters for this problem.

Creating New Information from College Data

Return to the `colleges` data. The variable `Cost` gives the listed annual cost of attendance, while `Net_Tuition` gives the average net tuition after financial aid.

We can use these variables to create a new quantity:

```
colleges |>
  mutate(Expected_Discount = (Cost - Net_Tuition) / Cost) |> head()
```

```
##      Name      City State Enrollment Private
## 1 Abilene Christian University Abilene TX      3524 Private
```

```

## 2 Academy of Art University San Francisco CA 6898 Private
## 3 Adelphi University Garden City NY 5307 Private
## 4 Adrian College Adrian MI 1781 Private
## 5 AdventHealth University Orlando FL 1166 Private
## 6 Agnes Scott College Decatur GA 966 Private
## Region Adm_Rate ACT_median ACT_Q1 ACT_Q3 Cost Net_Tuition
## 1 South West 0.5696 24 21 21 48046 16177
## 2 Far West NA NA NA NA 39735 31908
## 3 Mid East 0.7418 25 22 22 49008 24971
## 4 Great Lakes 0.6481 23 19 19 51626 14136
## 5 South East 0.8689 20 18 18 24338 15360
## 6 South East 0.7040 NA NA NA 52309 13239
## Avg_Fac_Salary PercentFemale PercentWhite PercentBlack PercentHispanic
## 1 69804 0.6118200 0.7946 0.0814 0.1635
## 2 76428 0.5731982 0.6791 0.0825 0.1534
## 3 111339 0.7211121 0.6669 0.1785 0.1292
## 4 72873 0.4221106 0.8861 0.0692 0.0318
## 5 69759 0.8251058 0.7622 0.1395 0.1338
## 6 82647 NA 0.6884 0.2299 0.0665
## PercentAsian FourYearComp_Males FourYearComp_Females Debt_median
## 1 0.0287 0.41157556 0.5283019 16000
## 2 0.1191 0.09157212 0.1325224 12877
## 3 0.0673 0.61146497 0.6998855 19500
## 4 0.0121 0.23209169 0.3319838 18468
## 5 0.0259 0.47619048 0.4132231 16646
## 6 0.0321 NA NA 22450
## Salary10yr_median Expected_Discount
## 1 43000 0.6633018
## 2 38600 0.1969800
## 3 58500 0.4904709
## 4 38600 0.7261845
## 5 56000 0.3688882
## 6 39400 0.7469078

```

The variable `Expected_Discount` measures the difference between listed cost and net tuition as a proportion of the listed cost.

Question #6

- A. Create a smaller data frame containing `Name`, `Cost`, `Net_Tuition`, and `Expected_Discount`.
- B. The variable `Debt_median` records median student debt. Create a new variable called `Debt_Net_Ratio` that compares median student debt to average net tuition:

$$\text{Debt Net Ratio} = \frac{\text{Median Student Debt}}{\text{Net Tuition}}.$$

- C. What would a value of `Debt_Net_Ratio` greater than 1 indicate?
- D. Create a data frame containing only the college name, net tuition, median student debt, and your new `Debt_Net_Ratio` variable.

Ordering Observations with `arrange()`

Sometimes the information we need is already present in a data frame, but the order of the observations makes it difficult to find what we are looking for.

The `arrange()` function changes the order of observations.

For example, we can arrange the penguins from smallest to largest body mass:

```
penguins |>
  arrange(body_mass_g) |> head()

## # A tibble: 6 x 8
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>           <int>         <int>
## 1 Chinstrap Dream         46.9           16.6             192          2700
## 2 Adelie   Biscoe         36.5           16.6             181          2850
## 3 Adelie   Biscoe         36.4           17.1             184          2850
## 4 Adelie   Biscoe         34.5           18.1             187          2900
## 5 Adelie   Dream         33.1           16.1             178          2900
## 6 Adelie   Torgersen        38.6            17             188          2900
## # i 2 more variables: sex <fct>, year <int>
```

By default, `arrange()` orders a quantitative variable from smallest to largest. We can use `desc()` to reverse this order:

```
penguins |>
  arrange(desc(body_mass_g)) |> head()

## # A tibble: 6 x 8
##   species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>           <int>         <int>
## 1 Gentoo Biscoe         49.2           15.2             221          6300
## 2 Gentoo Biscoe         59.6            17             230          6050
## 3 Gentoo Biscoe         51.1           16.3             220          6000
## 4 Gentoo Biscoe         48.8           16.2             222          6000
## 5 Gentoo Biscoe         45.2           16.4             223          5950
## 6 Gentoo Biscoe         49.8           15.9             229          5950
## # i 2 more variables: sex <fct>, year <int>
```

Unlike `filter()`, `arrange()` does not remove observations. Unlike `select()`, it does not remove variables. It simply changes the order in which the observations appear.

Question #7

- A. Arrange the `penguins` data so that the penguins with the longest flippers appear first.
- B. Return to the `murders` data. Using the murder rate per 100,000 residents, arrange the states from largest to smallest murder rate. Which state has the lowest prevalence of murders?
- C. Now arrange the original `murders` data from largest to smallest value of `total`. Is this the same state as Part B?
- D. Explain why ranking states by `total` and ranking states by `rate` answer different questions.

Putting the Pieces Together

We have now introduced four functions that change a data frame in different ways:

Function	Question to Consider
<code>filter()</code>	Which observations are relevant?
<code>select()</code>	Which variables are useful?
<code>mutate()</code>	Is there information I need to create?
<code>arrange()</code>	Would a particular order make the data easier to use?

These are useful questions to ask when working with data, but they are **not** a sequence of steps that must always be performed. A particular task may require some of these operations and not others, and they will not always occur in the same order.

Question #8

Suppose we are interested in comparing murder rates among states in the **Northeast**.

- Begin by creating a data frame containing only states in the Northeast.
- Retain only the state name, population, and total number of murders.
- Create a variable called **rate** containing the number of murders per 100,000 residents.
- Arrange the resulting data so that the states with the largest murder rates appear first.
- Now combine Parts A–D into a single pipeline if you haven’t already done so.
- What does the first row tell you?
- Suppose we removed the `select()` step from the pipeline. Could we still answer the original question? What would change about the resulting data frame?

Deciding What the Data Need

For the final question, we will work with a different dataset. The **gapminder** data contain information about countries around the world at several points in time:

```
head(gapminder)
```

```
## # A tibble: 6 x 6
##   country    continent  year lifeExp      pop gdpPercap
##   <fct>      <fct>    <int>  <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
## 6 Afghanistan Asia      1977   38.4 14880372    786.
```

Some of the variables are:

Variable	Meaning
<code>country</code>	Country
<code>continent</code>	Continent
<code>year</code>	Year
<code>lifeExp</code>	Life expectancy
<code>pop</code>	Population
<code>gdpPercap</code>	GDP per person

Question #9

A. What does one observation in `gapminder` represent? Be as precise as possible.

B. Suppose we want to investigate the following question: **Which Asian countries had the largest total economies in 2007?**

The dataset contains GDP per person and population, but it does **not** directly contain total GDP.

Construct a data frame that would help answer this question. Your result should contain only information relevant to the investigation and should be organized so that countries with the largest total GDP are easy to identify.

Unlike the previous question, you will need to decide which data manipulation operations are useful.

C. Which country had the largest total GDP according to your resulting data?

D. Briefly describe how you transformed the original `gapminder` data and why those changes were useful for answering the question.

Wrapping Up

Today we used `dplyr` to change which observations and variables were included in a data frame, create new information from existing variables, and reorganize observations so that useful information was easier to identify.

The appropriate manipulation depended on what we wanted to learn from the data. Before manipulating a dataset, we need to understand what its observations and variables represent and then determine how the current structure of the data differs from what our question requires.